LAPORAN TUGAS BESAR 1 IF3170
INTELIGENSI ARTIFISIAL



Dipersiapkan oleh:

13524021	Natanael Imandatua Manurung
13424102	Manuel Thimoty Silalahi
13524107	Rava Khoman Tuah Saragih

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

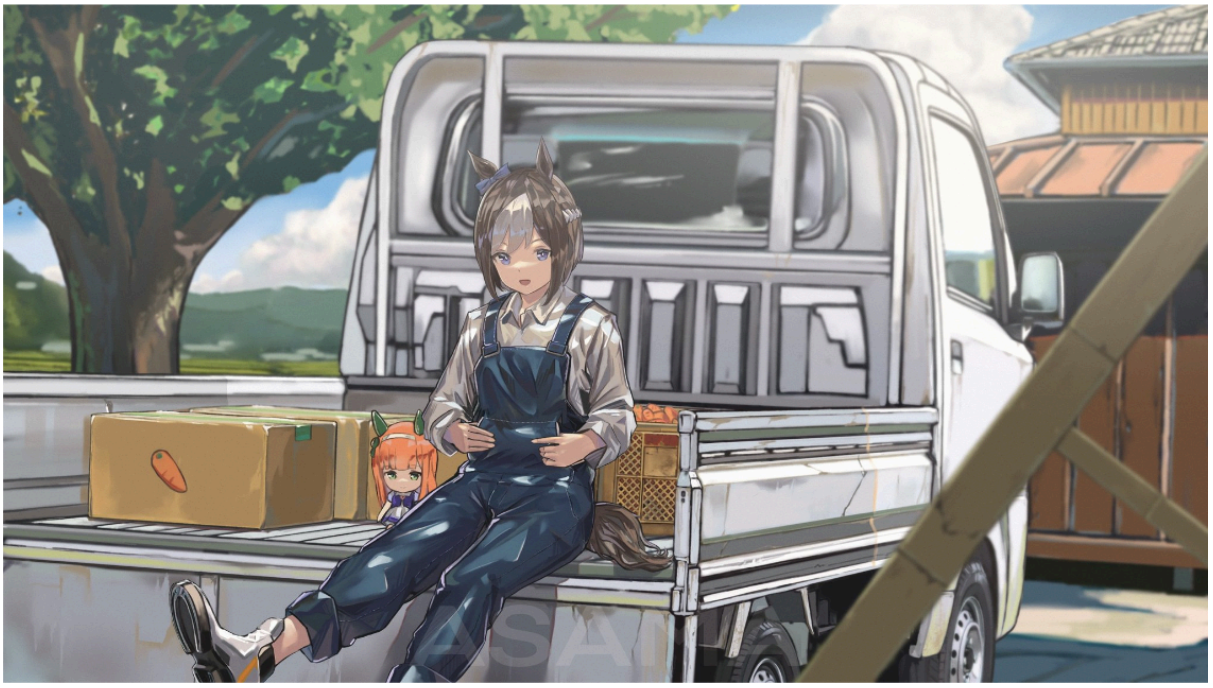
INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132 2026

Daftar Pustaka

Daftar Pustaka **2**
BAB 1
DESKRIPSI TUGAS **3**
BAB 2
PEMBAHASAN **5**
2.1. Representasi State 5
2.2. Objective Function dan Constraints 5
2.3. Hill-Climbing 7
2.3.1. Pembentukan State Tetangga 7
2.3.1.1. Menukar dua paket 7
2.3.1.2. Memindahkan paket 7
2.3.1.3. Merotasi paket 8
2.3.2. Steepest-Ascent Hill-Climbing 9
2.3.3. Sideways Hill-Climbing 10
2.3.4. Stochastic Hill-Climbing 12
2.3.5. Random-Restart Hill-Climbing 13
2.4. Simulated Annealing 15
Simulated Annealing (SA) adalah algoritma local search yang terinspirasi dari proses annealing pada metalurgi: logam dipanaskan lalu didinginkan perlahan agar mencapai struktur yang stabil. Hill-climbing hanya menerima neighbor yang lebih baik, sehingga mudah terjebak di local optima. SA mengatasinya dengan kadang-kadang menerima neighbor yang lebih buruk, dan peluang itu mengecil seiring turunnya "suhu" (temperature) T. 15
BAB 3 KESIMPULAN DAN SARAN **17**
BAB 4 PEMBAGIAN TUGAS **18**
BAB 5 LAMPIRAN **19**

BAB 1
DESKRIPSI TUGAS



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の銀堡垒🐴C108土曜日 東地区ホ-10a on X)

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk ujicoba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan "Lebih kilat dari Umamusume".

Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

Permasalahan yang diselesaikan adalah menentukan paket mana yang dimasukkan ke dalam truk, posisi setiap paket, serta orientasi paket tersebut. Konfigurasi harus memenuhi batasan kapasitas dan dimensi truk. Selain itu, paket tidak boleh saling bertumpang tindih, tidak boleh menembus dinding peti kemas, dan tidak boleh melayang. Paket pecah-belah juga tidak boleh menjadi penopang bagi paket lain.

Tujuan utama optimasi adalah memaksimalkan total nilai paket yang berhasil dimuat ke dalam truk. Paket yang tidak dapat dimuat tetap direpresentasikan sebagai paket yang berada di luar truk.

BAB 2

PEMBAHASAN

2.1. Representasi State

Satu state merepresentasikan konfigurasi lengkap sebuah truk beserta seluruh paket yang tersedia. Struktur state terdiri atas:

1. Informasi truk, meliputi:
   - dimensinya (lebar, panjang, dan tinggi)
   - kapasitas maksimum berdasarkan berat.
2. Informasi setiap paket, meliputi:
   - id
   - dimensi asli paket
   - orientasi
   - nilai paket
   - berat paket
   - status pecah-belah
   - ETA
   - posisi paket

Posisi paket direpresentasikan menggunakan koordinat bilangan bulat (x, y, z). Koordinat tersebut menunjukkan posisi sudut kiri-bawah-belakang paket terhadap sistem koordinat peti kemas.
Jika paket berada di luar truk, posisi paket direpresentasikan dengan nilai None. Dengan demikian, seluruh paket tetap tercatat di dalam state, baik paket yang berhasil dimuat maupun paket yang tidak dimuat.

Orientasi paket direpresentasikan sebagai permutasi sumbu x, y, dan z. Sebagai contoh, orientasi ("y", "x", "z") berarti sumbu lebar paket diarahkan ke sumbu y, sumbu panjang diarahkan ke sumbu x, dan sumbu tinggi tetap diarahkan ke sumbu z. Terdapat enam kemungkinan orientasi untuk setiap paket.
Pada proses inisialisasi, seluruh paket mula-mula berada di luar truk. Kemudian, paket-paket dipilih secara acak untuk dicoba dimasukkan ke dalam truk dengan orientasi, posisi horizontal, dan urutan yang acak. Posisi vertikal paket ditentukan pada ketinggian terendah yang memungkinkan paket tetap berada dalam konfigurasi valid.

2.2. Objective Function dan Constraints

Objective function yang digunakan adalah total nilai seluruh paket yang berada di dalam truk, dihitung dengan:

$$f(s) = \sum_{p \in P_{inside}(s)} value(p)$$

- s adalah state;
- $P_{inside}(s)$ adalah himpunan paket yang berada di dalam truk;
- $value(p)$ adalah nilai paket p.

Semakin besar nilai objective function, semakin baik konfigurasi tersebut. Paket yang berada di luar truk tidak berkontribusi terhadap nilai objective function.

Suatu state hanya dianggap valid jika memenuhi constraint berikut.

2.2.1. Tidak menembus dinding truk

Seluruh bagian paket harus berada di dalam batas dimensi peti kemas:

$$0 \leq x, 0 \leq y, 0 \leq z$$

$$x + w \leq W, y + l \leq L, z + h \leq H$$

2.2.2. Tidak terdapat paket yang bertumpang tindih

Dua paket di dalam truk tidak boleh menempati volume yang sama.

2.2.3. Paket tidak boleh melayang

Paket harus berada langsung di atas lantai truk atau memiliki paket lain sebagai penopang. Paket dianggap memiliki penopang apabila bagian atas paket lain tepat berada di bawah paket tersebut dan memiliki irisan secara horizontal.

2.2.4. Paket pecah-belah tidak boleh menjadi penopang

Apabila sebuah paket ditandai sebagai pecah-belah, tidak boleh ada paket lain yang diletakkan tepat di atasnya.

2.2.5. Berat total tidak boleh melebihi kapasitas truk

$$\sum_{p \in P_{inside}(s)} weight(p) \leq MaxCapacity$$

State yang melanggar salah satu constraint tidak digunakan sebagai kandidat solusi. Dalam implementasi objective function, state tidak valid diberi nilai $-\infty$, sehingga tidak akan dipilih oleh algoritma optimasi.

2.3. Hill-Climbing

Hill-climbing merupakan algoritma local search yang memulai pencarian dari sebuah initial state. Pada setiap iterasi, algoritma membentuk beberapa state tetangga (neighbor state), kemudian berpindah ke tetangga yang dianggap lebih baik berdasarkan nilai objective function.

Algoritma ini hanya menyimpan current state dan kandidat tetangganya. Oleh karena itu, kebutuhan memorinya relatif kecil dibandingkan algoritma pencarian yang menyimpan seluruh jalur eksplorasi.

Pada permasalahan pengisian paket, kualitas sebuah state ditentukan berdasarkan total nilai paket yang berhasil dimasukkan ke dalam truk. State dengan total nilai lebih besar dianggap lebih baik selama seluruh constraint terpenuhi.

2.3.1. Pembentukan State Tetangga

State tetangga dibentuk menggunakan tiga jenis operasi yang sesuai dengan spesifikasi permasalahan.

2.3.1.1. Menukar dua paket

Operasi ini menukar posisi dua paket. Jika salah satu paket berada di luar truk, maka paket tersebut dapat bertukar posisi dengan paket yang berada di dalam truk.

```
def swap_packages(state: State, first_id: str, second_id: str) ->
State:
    if first_id == second_id:
        return state.copy()
    first, second = state[first_id], state[second_id]
    result = state.copy()
    first_copy, second_copy = first.copy(), second.copy()
        first_copy.position, second_copy.position = second.position,
first.position
    result.packages[first_id] = first_copy
    result.packages[second_id] = second_copy
    return result
```

2.3.1.2. Memindahkan paket

Sebuah paket dapat dipindahkan ke koordinat integer lain di dalam truk. Paket juga dapat dipindahkan ke luar truk dengan menetapkan posisi None.

```
def move_package(state: State, package_id: str, position: tuple[int,
int, int] | None) -> State:
```

```
    result = state.copy()
    result.packages[package_id].position = position
    return result
```

Posisi yang mungkin dicoba adalah seluruh koordinat yang masih berada dalam batas truk, ditambah posisi None yang menunjukkan bahwa paket berada di luar truk.

2.3.1.3. Merotasi paket

Paket dapat diputar 90 derajat terhadap sumbu x, y, atau z. Rotasi dilakukan dengan menukar dua sumbu selain sumbu rotasi.

```
def rotate_orientation(orientation: tuple[str, str, str], axis: str) ->
tuple[str, str, str]:

    first, second = (item for item in AXES if item != axis)

    swap = {first: second, second: first}

    return tuple(swap.get(item, item) for item in orientation)



def   rotate_package(state:   State,   package_id:   str,   orientation:
tuple[str, str, str]) -> State:

    result = state.copy()

    result.packages[package_id].orientation = orientation

    return result
```

Setiap kandidat yang dibentuk kemudian diperiksa menggunakan ConstraintChecker. Kandidat yang melanggar batas dimensi truk, bertumpang tindih, melayang, menjadikan paket pecah-belah sebagai penopang, atau melebihi kapasitas truk akan dibuang.

```
def add(candidate: State) -> bool:
    signature = _signature(candidate)
    if signature in seen or not checker.is_valid(candidate):
        return False
    seen.add(signature)
    result.append(candidate)
    return max_neighbors is not None and len(result) >= max_neighbors
```

Dengan mekanisme ini, algoritma hanya melakukan pencarian pada state yang valid.

2.3.2. Steepest-Ascent Hill-Climbing

Varian utama yang digunakan adalah steepest-ascent hill-climbing. Pada setiap iterasi, seluruh tetangga valid dari state saat ini dievaluasi. Tetangga dengan nilai objective function terbesar dipilih sebagai kandidat perpindahan.

Implementasi inti algoritmanya adalah sebagai berikut:

```
class SteepestAscentHillClimbing:
    def __init__(
        self,
        objective_fn: ObjectiveFunction | None = None,
        neighbor_fn: NeighborFunction | None = None,
        checker: ConstraintChecker | None = None,
        max_iterations: int = 1_000,
    ):
        self._checker = checker
                self._objective = objective_fn or (lambda state:
objective(state, checker))
                 self._neighbors = neighbor_fn or (lambda state:
enumerate_neighbors(state, checker))
        self.max_iterations = max_iterations

    def run(self, initial_state: State) -> HillClimbingResult:
        start = perf_counter()
        initial = initial_state.copy()
        current = initial.copy()
        current_value = self._objective(current)
        history = [current_value]
        iterations = 0
        stop_reason = "local_optimum"

        while iterations < self.max_iterations:
            neighbors = list(self._neighbors(current))
            if not neighbors:
                stop_reason = "no_valid_neighbors"
                break
            best = max(neighbors, key=self._objective)
            best_value = self._objective(best)
            if best_value <= current_value:
                stop_reason = "local_optimum"
```

```
                break
            current = best.copy()
            current_value = best_value
            history.append(current_value)
            iterations += 1
        else:
            stop_reason = "max_iterations"

        return HillClimbingResult(
            initial_state=initial,
            final_state=current,
            history=history,
            iterations=iterations,
            duration_seconds=perf_counter() - start,
            stop_reason=stop_reason,
        )
```

Urutan prosesnya adalah:

1. Menetapkan initial state sebagai current state.
2. Menghitung nilai objective function initial state.
3. Membentuk seluruh state tetangga yang valid.
4. Memilih tetangga dengan objective function terbesar.
5. Membandingkan nilai tetangga terbaik dengan current state.
6. Berpindah ke tetangga tersebut jika nilainya lebih besar.
7. Menghentikan pencarian apabila tidak terdapat tetangga yang lebih baik.
8. Menghentikan pencarian apabila jumlah iterasi mencapai batas maksimum.

Keunggulan varian ini adalah setiap perpindahan dipilih secara terarah menuju peningkatan nilai terbesar pada lingkungan lokal. Namun, seluruh tetangga harus dievaluasi pada setiap iterasi. Akibatnya, waktu eksekusi dapat meningkat apabila jumlah paket atau ukuran truk semakin besar.

Selain itu, algoritma tidak menerima perpindahan menuju state yang lebih buruk. Akibatnya, algoritma dapat berhenti pada local optimum, meskipun masih terdapat solusi yang lebih baik di wilayah pencarian lain.

2.3.3. Sideways Hill-Climbing

Sideways hill-climbing merupakan modifikasi dari steepest-ascent hill-climbing yang mengizinkan algoritma berpindah ke tetangga dengan nilai objective function yang sama.

Perpindahan dengan nilai sama berguna untuk melewati area datar atau plateau. Tanpa mekanisme ini, algoritma akan langsung berhenti ketika tidak menemukan tetangga yang lebih baik.

```
class SidewaysHillClimbing(SteepestAscentHillClimbing):
    def __init__(self, *args, max_sideways: int = 10, **kwargs):
        super().__init__(*args, **kwargs)
        self.max_sideways = max_sideways

    def run(self, initial_state: State) -> HillClimbingResult:
        start = perf_counter()
        initial = initial_state.copy()
        current = initial.copy()
        current_value = self._objective(current)
        history = [current_value]
        iterations = 0
        sideways = 0
        stop_reason = "local_optimum"
        while iterations < self.max_iterations:
            neighbors = list(self._neighbors(current))
            if not neighbors:
                stop_reason = "no_valid_neighbors"
                break
            best = max(neighbors, key=self._objective)
            best_value = self._objective(best)
            if best_value < current_value:
                stop_reason = "local_optimum"
                break
            if best_value == current_value:
                if sideways >= self.max_sideways:
                    stop_reason = "max_sideways"
                    break
                sideways += 1
            current = best.copy()
            current_value = best_value
            history.append(current_value)
            iterations += 1
        else:
            stop_reason = "max_iterations"

        return HillClimbingResult(
            initial_state=initial,
            final_state=current,
            history=history,
```

```
            iterations=iterations,
            duration_seconds=perf_counter() - start,
            stop_reason=stop_reason,
            sideways_moves=sideways,
        )
```

Parameter max_sideways digunakan untuk membatasi jumlah perpindahan dengan nilai objective function yang sama. Pada implementasi ini, jumlah sideways move dihitung secara kumulatif. Jika batas tersebut tercapai, pencarian dihentikan untuk mencegah algoritma berputar terlalu lama pada area datar.

Keunggulannya adalah peluang untuk keluar dari plateau lebih besar dibandingkan steepest-ascent biasa. Kekurangannya adalah algoritma dapat melakukan beberapa iterasi tanpa meningkatkan nilai objective function.

2.3.4. Stochastic Hill-Climbing

Pada stochastic hill-climbing, algoritma tidak selalu memilih tetangga terbaik. Algoritma terlebih dahulu mengumpulkan tetangga yang memiliki nilai objective function lebih besar daripada current state, kemudian memilih salah satunya secara acak.

```
def stochastic_hill_climbing(
    initial_state: State,
    rng: random.Random | None = None,
    max_iterations: int = 1000,
) -> SearchResult:
    rng = rng or random.Random()
    checker = ConstraintChecker(initial_state.truck)
    current = initial_state
    history = [current.total_value]
    improvements = 0
    for _ in range(max_iterations):
        neighbor = random_neighbor(current, checker, rng)
                if neighbor is not None and neighbor.total_value >
current.total_value:
            current = neighbor
            improvements += 1
        history.append(current.total_value)

    return SearchResult(
        algorithm="stochastic_hill_climbing",
        initial_state=initial_state,
```

```
        final_state=current,
        initial_objective=objective(initial_state, checker),
        final_objective=objective(current, checker),
        iterations=max_iterations,
        history=history,
        params={"max_iterations": max_iterations},
        extra={"improvements": improvements},
    )
```

Terdapat dua mode pembentukan tetangga:

- full: seluruh tetangga valid dievaluasi;
- sample: hanya sejumlah tetangga acak yang diambil.

Mode sample dapat mengurangi waktu eksekusi karena tidak perlu mengevaluasi seluruh kemungkinan posisi, rotasi, dan pertukaran paket. Namun, tetangga terbaik mungkin tidak ikut terambil dalam sampel.

Keunggulan stochastic hill-climbing adalah pencarian tidak terlalu deterministik dan dapat mengeksplorasi jalur yang berbeda. Kekurangannya, hasil akhir dapat lebih tidak konsisten karena bergantung pada pilihan acak di setiap iterasi.

2.3.5. Random-Restart Hill-Climbing

Random-restart hill-climbing menjalankan algoritma hill-climbing berkali-kali dengan initial state yang berbeda. Setiap proses pencarian menghasilkan local optimum masing-masing. Setelah seluruh restart selesai, solusi dengan objective function terbesar dipilih sebagai solusi akhir.

```
def random_restart_hill_climbing(
    initial_state: State,
    rng: random.Random | None = None,
    max_restart: int = 10,
    inner: HillClimber = stochastic_hill_climbing,
    inner_params: Mapping[str, Any] | None = None,
) -> SearchResult:
    rng = rng or random.Random()
    inner_params = dict(inner_params or {})
    packages = list(initial_state)

    best: SearchResult | None = None
    history: list[float] = []
    iterations_per_restart: list[int] = []
```

```
    objective_per_restart: list[float] = []
    restart_starts: list[int] = []
    restarts = 0
    start = initial_state
    while True:
        result = inner(start, rng, **inner_params)
        restart_starts.append(len(history))
        history.extend(result.history)
        iterations_per_restart.append(result.iterations)
        objective_per_restart.append(result.final_objective)
                    if  best  is  None  or  result.final_objective  >
best.final_objective:
            best = result
        if restarts >= max_restart:
            break
        restarts += 1
           start  =  random_initial_state(initial_state.truck,  packages,
rng)

    assert best is not None
    return SearchResult(
        algorithm="random_restart_hill_climbing",
        initial_state=initial_state,
        final_state=best.final_state,
                              initial_objective=objective(initial_state,
ConstraintChecker(initial_state.truck)),
        final_objective=best.final_objective,
        iterations=sum(iterations_per_restart),
        history=history,
                  params={"max_restart":  max_restart,  "inner":  inner,
"inner_params": inner_params},
        extra={
            "restarts": restarts,
            "iterations_per_restart": iterations_per_restart,
            "objective_per_restart": objective_per_restart,
            "restart_starts": restart_starts,
                                                        "best_restart":
objective_per_restart.index(best.final_objective),
        },
    )
```

Pada implementasi ini, initial state pertama diproses terlebih dahulu, kemudian algoritma melakukan restart menggunakan initial state acak baru. Dengan demikian, apabila max_restart = 10, terdapat satu pencarian awal dan hingga sepuluh restart tambahan.

Keunggulan random-restart hill-climbing adalah kemampuannya mengurangi ketergantungan terhadap satu initial state. Jika salah satu pencarian terjebak pada local optimum yang buruk, pencarian lain masih berpeluang menemukan konfigurasi yang lebih baik.

Kekurangannya adalah waktu eksekusi meningkat seiring bertambahnya jumlah restart. Selain itu, hasil pencarian tetap bergantung pada kualitas pembangkitan initial state secara acak.

2.4. Simulated Annealing

Simulated Annealing (SA) adalah algoritma local search yang terinspirasi dari proses annealing pada metalurgi: logam dipanaskan lalu didinginkan perlahan agar mencapai struktur yang stabil. Hill-climbing hanya menerima neighbor yang lebih baik, sehingga mudah terjebak di local optima. SA mengatasinya dengan kadang-kadang menerima neighbor yang lebih buruk, dan peluang itu mengecil seiring turunnya "suhu" (temperature) T.
Untuk persoalan pengisian peti kemas ini, SA bekerja sebagai berikut:

- State: satu konfigurasi lengkap, yaitu posisi dan orientasi setiap paket (di dalam maupun di luar truk).
- Nilai state (objective function, dimaksimalkan): total value paket di dalam truk. State yang melanggar constraint tidak pernah dikunjungi, sehingga nilai penalti tidak diperlukan (lihat bagian 2).
- Neighbor: satu langkah acak dari tiga jenis yang diizinkan spesifikasi: menukar posisi dua paket, memindahkan satu paket ke koordinat lain (atau ke luar truk), atau merotasi satu paket 90° terhadap sumbu x, y, atau z.
- Keputusan pindah state: dengan $\Delta E = value(neighbo) - value(current\ state)$
    - $\Delta E \geq 0$: neighbor selalu diterima.
    - $\Delta E \leq 0$: neighbor diterima dengan peluang $e^{\frac{\Delta E}{T}}$.

Karena $\Delta E \leq 0$, nilai $e^{\frac{\Delta E}{T}}$ berada di (0, 1]. Pada T besar peluangnya mendekati 1 (hampir semua langkah diterima, pencarian bersifat penjelajahan). Pada T kecil peluangnya mendekati 0 (hampir hanya langkah yang lebih baik diterima, perilaku menyerupai hill-climbing).

```
def simulated_annealing(
```

```
    initial_state: State,
    rng: random.Random | None = None,
    initial_temperature: float | None = None,
    cooling_rate: float = 0.999,
    min_temperature: float = 1e-3,
    max_iterations: int = 20000,
    stuck_window: int = 200,
    neighbor_attempts: int = 40,
) -> SearchResult:
    rng = rng or random.Random()
    if not 0 < cooling_rate < 1:
        raise ValueError("cooling_rate must be in (0, 1)")
    checker = ConstraintChecker(initial_state.truck)
    t0 = initial_temperature if initial_temperature is not None else
estimate_initial_temperature(initial_state, rng)

    current = initial_state
    current_value = current.total_value
    best, best_value = current, current_value

    history = [current_value]
    best_history = [best_value]
    temperatures = [t0]
    acceptance = []
    accepted_better=accepted_worse = rejected_worse = sideways = no_neighbor = 0
    stuck_count, since_improve, stuck_events = 0, 0, []

    iteration = 0
    temperature = t0
    while iteration < max_iterations:
        temperature = t0*cooling_rate**(iteration + 1)
        if temperature<min_temperature:
            break
        iteration +=1

        neighbor = random_neighbor(current, checker, rng,
attempts=neighbor_attempts)
        if neighbor is None:
            no_neighbor +=1
        else:
            delta = neighbor.total_value - current_value
            if delta > 0:
                accepted = True
                accepted_better +=1
            else:
                probability = math.exp(delta / temperature)
                accepted = rng.random() < probability
                if delta ==0:
                    sideways+=accepted
```

```
                else:
                    acceptance.append([iteration, temperature, delta,
probability, accepted])
                    if accepted:
                        accepted_worse+= 1
                    else:
                        rejected_worse += 1
            if accepted:
                current, current_value = neighbor, neighbor.total_value

        if current_value > best_value:
            best, best_value = current, current_value
            since_improve = 0
        else:
            since_improve += 1
            if since_improve == stuck_window:
                stuck_count += 1
                stuck_events.append(iteration)

        history.append(current_value)
        best_history.append(best_value)
        temperatures.append(temperature)
```

Setiap run SA mencatat:

- state awal dan state akhir (best), serta nilai **objective function** keduanya;
- riwayat nilai state saat ini (history) dan nilai terbaik sejauh ini (best_history) tiap iterasi, untuk plot OF terhadap iterasi;
- **Untuk setiap langkah dengan** $\Delta E < 0$: iterasi, suhu, ΔE, nilai $e^{\frac{\Delta E}{T}}$, dan apakah langkah itu diterima (`acceptance`)
- frekuensi stuck (stuck_count) dan iterasi terjadinya (stuck_events);
- durasi proses pencarian dan banyak iterasi;
- statistik pendukung: accepted_better, accepted_worse, rejected_worse, sideways, no_neighbor.

Catatan: langkah dengan $\Delta E = 0$ *(nilai sama) memiliki* $e^{\frac{\Delta E}{T}} = 1$*, sehingga selalu diterima; langkah ini dihitung terpisah sebagai `sideways`*

2.5. Genetic Algorithm

Genetic Algorithm (GA) merupakan algoritma pencarian yang mengembangkan sekumpulan kandidat solusi melalui proses seleksi, crossover, dan mutasi. Berbeda dengan hill-climbing dan simulated

annealing yang memperbarui satu state pada setiap iterasi, GA mempertahankan sejumlah state dalam sebuah populasi.

Pada permasalahan pengisian paket, setiap individu merepresentasikan satu konfigurasi lengkap truk. Individu dengan nilai fitness lebih tinggi memiliki peluang lebih besar untuk dipilih sebagai parent. Konfigurasi dari dua parent kemudian dikombinasikan untuk membentuk child, yang dapat mengalami mutasi sebelum dimasukkan ke populasi berikutnya.

2.5.1. Representasi Individu dan Inisialisasi Populasi

Satu individu direpresentasikan menggunakan objek State yang memuat informasi truk dan seluruh paket. Setiap gen berkaitan dengan satu paket, terutama posisi dan orientasinya. Posisi bernilai None menunjukkan bahwa paket berada di luar truk. Atribut asli paket, seperti dimensi, value, berat, status pecah-belah, dan ETA, tetap dipertahankan.

Urutan gen mengikuti urutan ID paket dalam state. Dengan demikian, crossover menggabungkan konfigurasi paket dari dua parent tanpa menghilangkan atau menggandakan identitas paket.

Populasi awal dibentuk dengan memanggil random_initial_state sebanyak population_size. Setiap individu dibangkitkan secara acak menggunakan data truk dan paket yang sama.

```
population = [
    random_initial_state(initial_state.truck, initial_state, rng,
                         fill_prob=initial_fill_prob, attempts=initial_attempts)
    for _ in range(population_size)
]
```

Parameter initial_fill_prob menentukan peluang paket dicoba untuk dimasukkan, sedangkan initial_attempts membatasi percobaan penempatan pada proses inisialisasi. Pembentukan populasi menggunakan generator bilangan acak yang dapat dikendalikan melalui seed agar eksperimen dapat diulang.

State awal yang dicatat pada hasil GA adalah individu dengan fitness tertinggi pada generasi 0. State tersebut menjadi representasi hasil inisialisasi, bukan satu-satunya titik awal pencarian, karena GA memproses seluruh populasi.

2.5.2. Fitness dan Seleksi Parent

Fitness utama menggunakan objective function yang sama dengan algoritma lain, yaitu total value paket yang berada di dalam truk. Seluruh individu harus memenuhi constraint sebelum digunakan sebagai kandidat solusi.

Pemilihan parent menggunakan seleksi roulette atau fitness-proportionate selection. Peluang individu ke-i untuk terpilih dihitung berdasarkan perbandingan fitness individu tersebut terhadap total fitness populasi:

$$P(i) = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

dengan (f_i) merupakan fitness individu ke-i dan (N) merupakan jumlah individu dalam populasi.

Implementasinya adalah sebagai berikut:

```
def _select_parent(population: list[State], fitness: list[float], rng: random.Random) -> State:
    if not any(fitness):
        return rng.choice(population)
    return rng.choices(population, weights=fitness, k=1)[0]
```

Jika seluruh fitness bernilai nol, parent dipilih secara acak dengan peluang yang sama. Seleksi dilakukan dua kali untuk menghasilkan sepasang parent. Individu yang sama dapat terpilih lebih dari satu kali, termasuk sebagai kedua parent dalam satu pasangan.

Seleksi roulette membutuhkan fitness yang berhingga dan tidak negatif. Oleh karena itu, implementasi memeriksa nilai fitness sebelum proses seleksi dilakukan. Individu dengan fitness tinggi lebih berpeluang terpilih, tetapi individu dengan fitness rendah tetap memiliki kesempatan untuk menghasilkan keturunan.

2.5.3. Crossover

Crossover menggunakan metode satu titik atau single-point crossover. Algoritma memilih satu titik potong pada urutan ID paket. Paket sebelum titik potong disalin dari parent pertama, sedangkan paket setelah titik potong disalin dari parent kedua.

```
for _ in range(attempts):
    point = rng.randrange(1, len(ids))
    packages = [first[package_id].copy() for package_id in ids[:point]]
    packages.extend(second[package_id].copy() for package_id in ids[point:])
    child = State(first.truck, packages)
    if checker.is_valid(child):
        return child
return rng.choice((first, second)).copy()
```

Meskipun kedua parent valid, child yang dihasilkan belum tentu valid. Sebagai contoh, paket yang disalin dari parent pertama dapat bertumpang tindih dengan paket dari parent kedua. Paket juga dapat kehilangan penopang karena konfigurasi paket di bawahnya berasal dari parent yang berbeda.

Setiap child diperiksa menggunakan ConstraintChecker. Jika tidak valid, algoritma mencoba titik potong lain hingga batas crossover_attempts. Apabila seluruh percobaan gagal, algoritma menggunakan salinan salah satu parent sebagai child. Untuk persoalan dengan kurang dari dua paket, crossover juga langsung menghasilkan salinan salah satu parent.

Mekanisme ini menjaga validitas populasi tanpa melakukan perbaikan posisi secara langsung. Namun, apabila banyak crossover gagal, populasi berikutnya dapat didominasi salinan parent sehingga keragamannya berkurang.

2.5.4. Mutasi

Mutasi dilakukan pada child dengan peluang sebesar mutation_rate. Operasi mutasi menggunakan tiga perubahan yang diperbolehkan dalam spesifikasi:

1. Menukar posisi dua paket.
2. Memindahkan satu paket ke koordinat lain atau ke luar truk.
3. Merotasi satu paket 90 derajat terhadap sumbu x, y, atau z.

Jenis operasi dipilih secara acak. Untuk operasi pemindahan, kandidat posisi diambil dari koordinat integer yang memungkinkan paket berada dalam batas dimensi truk, ditambah posisi None. Kandidat tetap harus diperiksa karena posisi tersebut dapat bertumpang tindih dengan paket lain, melayang, atau melanggar constraint lainnya.

Mutasi hanya diterima jika menghasilkan perubahan posisi atau orientasi dan state yang terbentuk valid.

```
changed = any(candidate[package.id].position != package.position
              or candidate[package.id].orientation != package.orientation for package in packages)
if changed and checker.is_valid(candidate):
    return candidate
```

Percobaan dibatasi oleh mutation_attempts. Jika tidak ditemukan perubahan yang valid, algoritma mengembalikan salinan state sebelum mutasi.

Mutasi tidak harus meningkatkan fitness. Selama valid, perubahan yang menurunkan fitness juga dapat diterima. Mekanisme ini memungkinkan terbentuknya konfigurasi baru yang tidak dihasilkan oleh crossover saja.

2.5.5. Pembentukan Generasi dan Kondisi Berhenti

Pada setiap generasi, algoritma menghitung fitness seluruh individu, kemudian mencatat fitness maksimum dan rata-rata populasi. Selanjutnya, algoritma membentuk populasi baru dengan jumlah individu yang sama.

```
next_population = []
for _ in range(population_size):
    first = _select_parent(population, fitness, rng)
    second = _select_parent(population, fitness, rng)
    child = _crossover(first, second, checker, rng, crossover_attempts)
    if rng.random() < mutation_rate:
        child = _mutate(child, checker, rng, mutation_attempts)
    next_population.append(child)
population = next_population
```

Urutan prosesnya adalah:

1. Membentuk populasi awal secara acak.
2. Menghitung fitness setiap individu.
3. Mencatat fitness maksimum dan rata-rata populasi.
4. Memilih dua parent menggunakan seleksi roulette.
5. Membentuk child melalui crossover satu titik.
6. Melakukan mutasi sesuai peluang yang ditentukan.
7. Mengulangi pembentukan child sampai populasi baru lengkap.
8. Mengganti seluruh populasi lama dengan populasi baru.
9. Menghentikan pencarian setelah jumlah generasi mencapai max_generations.

Implementasi ini tidak menggunakan elitism. Individu terbaik tidak otomatis dipertahankan pada generasi berikutnya. Akibatnya, fitness maksimum populasi dapat menurun apabila individu terbaik tidak diwariskan atau konfigurasinya berubah akibat crossover dan mutasi.

State akhir yang dikembalikan adalah individu terbaik dalam populasi terakhir, bukan individu terbaik sepanjang proses pencarian. Oleh karena itu, nilai objective akhir dapat lebih rendah daripada fitness terbaik pada generasi sebelumnya, termasuk generasi 0.

2.5.6. Pencatatan Hasil

Setiap run GA mencatat:

- State awal, yaitu individu terbaik pada generasi 0.

- State akhir, yaitu individu terbaik pada generasi terakhir.
- Nilai objective awal dan akhir.
- Fitness maksimum setiap generasi melalui history.
- Fitness rata-rata setiap generasi melalui mean_history.
- Jumlah populasi, jumlah generasi, laju mutasi, dan parameter percobaan operator.
- Seed atau keadaan generator bilangan acak.
- Durasi proses pencarian.

Histori mencakup generasi 0. Dengan demikian, run dengan G generasi menghasilkan G + 1 titik pada grafik.

Keunggulan GA adalah pencarian melibatkan banyak konfigurasi sekaligus. Crossover dapat menggabungkan susunan paket dari parent yang berbeda, sedangkan mutasi memberikan kesempatan untuk membentuk konfigurasi baru. Namun, constraint penempatan 3D membuat banyak hasil crossover tidak dapat digunakan. Selain itu, penggantian seluruh populasi tanpa elitism menyebabkan solusi yang baik dapat hilang. Hasil pencarian karena itu dipengaruhi oleh ukuran populasi, jumlah generasi, dan keberhasilan operator dalam mempertahankan keragaman konfigurasi.

2.6. Hasil dan Perbandingan

A. Dataset

Semua algoritma dijalankan pada dataset yang sama, yaitu tc.json. Berikut informasi sekilas terkait dataset :

Atribut	Nilai
Ukuran Peti Kemas	5 x 3 x 3 (volume = 45)
Kapasitas	24 unit berat
Jumlah Paket	14
Total Value Semua Paket	291
Total Berat Semua Paket	60 (melebihi kapasitas)

B. Global Optimum

Akan dihitung batas atas nilai objective dengan mencoba seluruh 2^{14} = 16.384 kombinasi paket dan memilih kombinasi bernilai tertinggi yang total berat nya kurang dari 24 dan total volume kurang dari 45. Hasilnya 144, dari paket D, F, G, H, J, K, N, R.

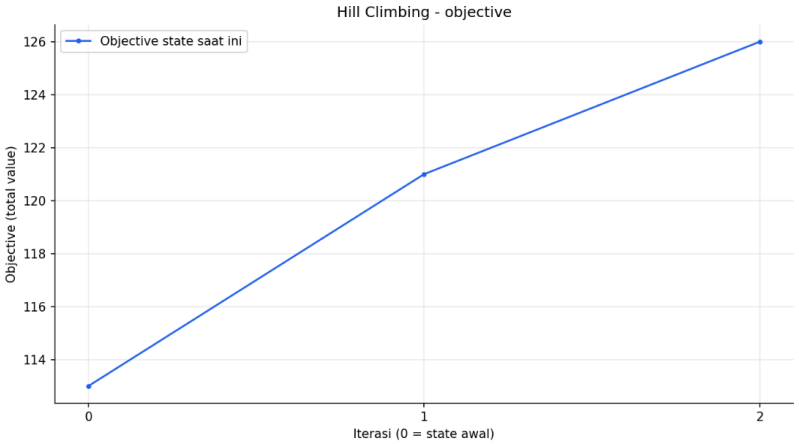
Simulated Annealing run 2 menghasilkan state akhir yang valid (lolos seluruh constraint) dengan tepat kedelapan paket tersebut di dalam truk, value 144, dan berat 24/24. Karena batas atas tercapai oleh state yang valid, global optimum dataset ini adalah 144. Nilai ini dipakai sebagai acuan di seluruh analisis.

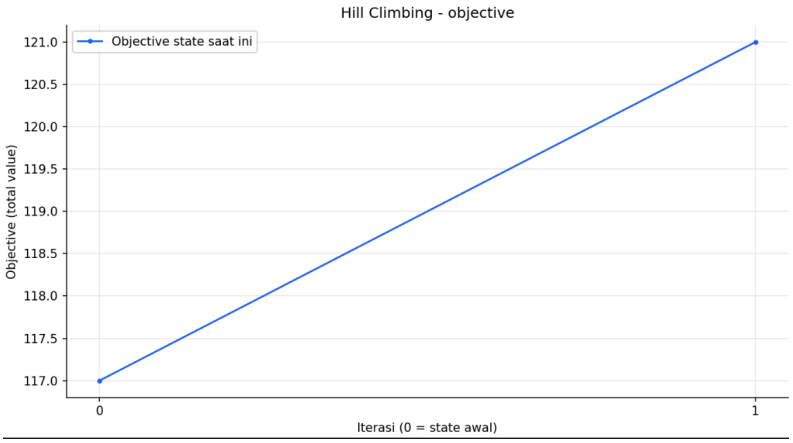
C. Prosedur

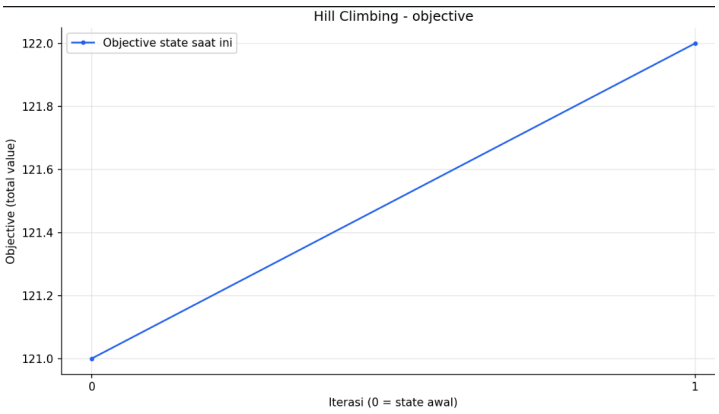
- Setiap algoritma selain GA dijalankan 3 kali melalui experiment.py. Run ke-i memakai seed i, sehingga kelima algoritma non-GA berangkat dari tiga initial state yang sama, dengan nilai objective 113, 117, dan 121. Perbedaan hasil antar-algoritma karena itu berasal dari algoritmanya, bukan dari titik awal.
- GA tidak memakai satu initial state melainkan populasi acak. Nilai "awal" GA adalah fitness terbaik pada generasi 0.

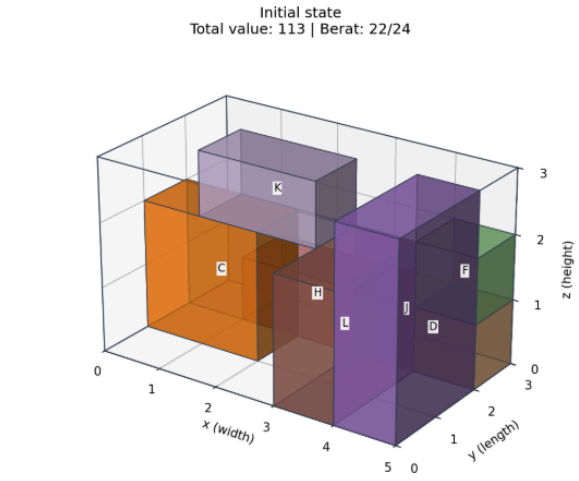
Algoritma	Parameter
Steepest Ascent HC	max_iterations = 100
HC with Sideways	max_sideways = 10 dan max_iterations = 1000
Stochastic HC	nmax = 1000
Random Restart HC	max_restart = 10
Simulated Annealing	alpha = 0.999, T_min = 0.0001, T0 otomatis, stuck_window = 200
Genetic Algorithm (3 run)	populasi = 30, generasi = 100, laju mutasi = 0.2, seleksi roulette, crossover 1 titik
Genetic Algorithm (variasi)	sesuai spesifikasi

D. Analisis Steepest Ascent Hill Climbing



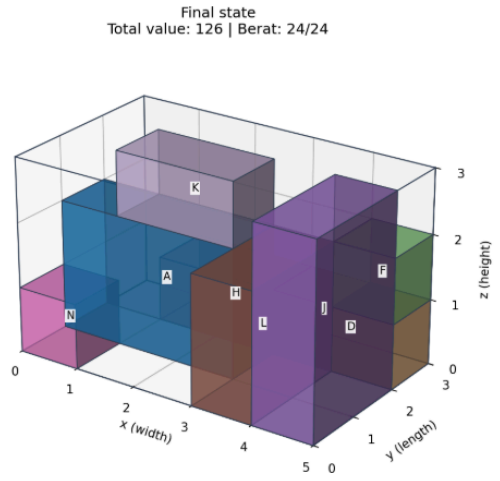






Di dalam: ID | posisi | dimensi efektif (x, y, z)
- C | (0, 1, 0) | (2, 1, 2)
- D | (3, 2, 0) | (2, 1, 1)
- F | (4, 2, 1) | (1, 1, 1)
- H | (1, 2, 0) | (2, 1, 1)
- J | (4, 0, 0) | (1, 2, 3)
- K | (1, 1, 2) | (2, 1, 1)
- L | (3, 0, 0) | (1, 2, 2)

Di luar (7): tanpa koordinat
- A | dimensi (3, 1, 2) | value 25
- B | dimensi (2, 1, 2) | value 18
- E | dimensi (2, 2, 2) | value 30
- G | dimensi (1, 3, 1) | value 17
- R | dimensi (2, 5, 1) | value 45
- M | dimensi (2, 3, 2) | value 35
- N | dimensi (1, 1, 1) | value 8

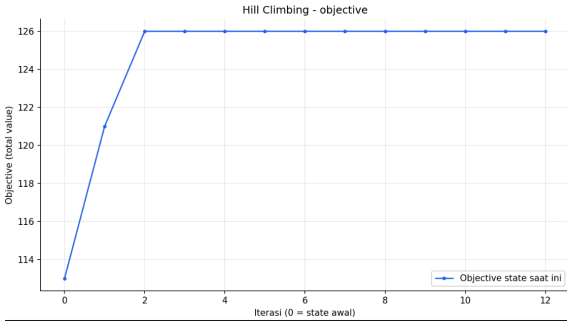


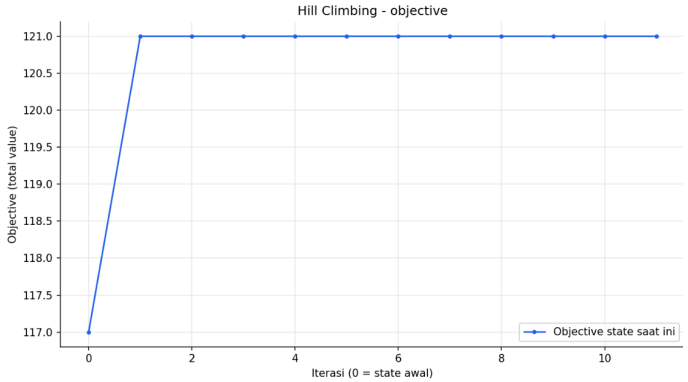
Di dalam: ID | posisi | dimensi efektif (x, y, z)
- A | (0, 1, 0) | (3, 1, 2)
- D | (3, 2, 0) | (2, 1, 1)
- F | (4, 2, 1) | (1, 1, 1)
- H | (1, 2, 0) | (2, 1, 1)
- J | (4, 0, 0) | (1, 2, 3)
- K | (1, 1, 2) | (2, 1, 1)
- L | (3, 0, 0) | (1, 2, 2)
- N | (0, 0, 0) | (1, 1, 1)

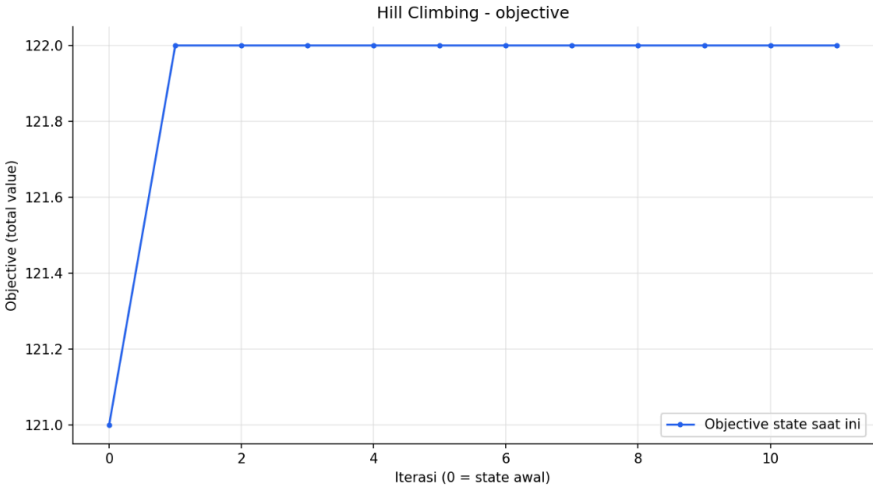
Di luar (6): tanpa koordinat
- B | dimensi (2, 1, 2) | value 18
- C | dimensi (2, 1, 2) | value 20
- E | dimensi (2, 2, 2) | value 30
- G | dimensi (1, 3, 1) | value 17
- R | dimensi (2, 5, 1) | value 45
- M | dimensi (2, 3, 2) | value 35

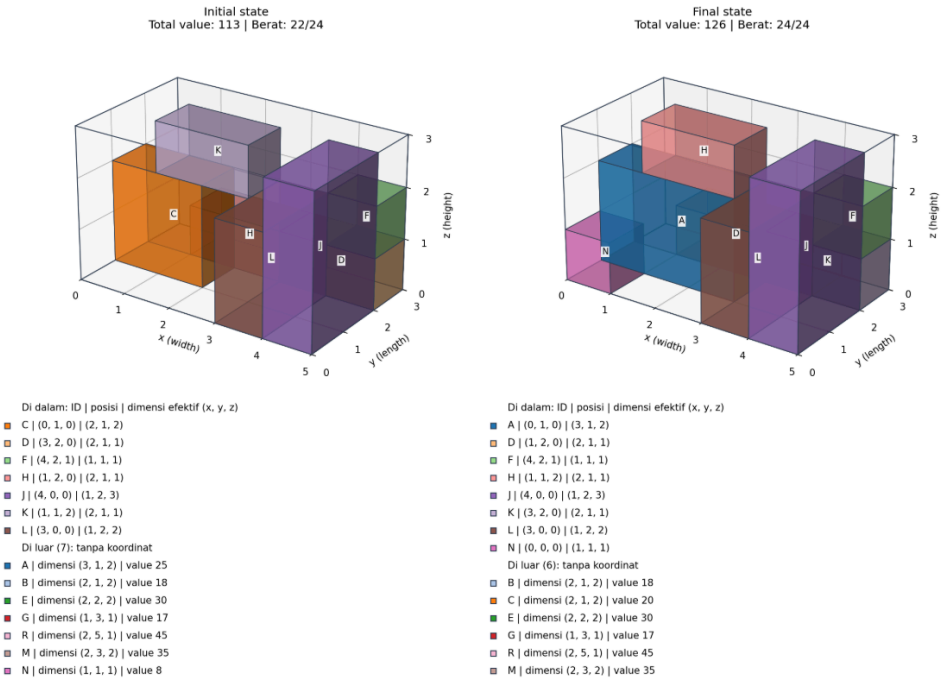
Analisis. Steepest Ascent berhenti sangat cepat, hanya 1–2 iterasi, dan selalu karena mencapai local optimum, tidak ada satupun neighbor yang lebih baik. Setiap iterasi memeriksa seluruh neighbor (semua swap, semua posisi grid untuk setiap paket, dan semua rotasi), sehingga langkah yang dipilih selalu yang terbaik. Namun pada truk yang sempit, setelah satu atau dua paket ditambahkan, ruang kosong yang tersisa tidak lagi muat untuk paket lain dengan satu langkah, sehingga pencarian terjebak. Hasil akhirnya berkisar 121–126, dan sangat ditentukan oleh initial state.

E. Analisis Hill Climbing with Sideaways Move



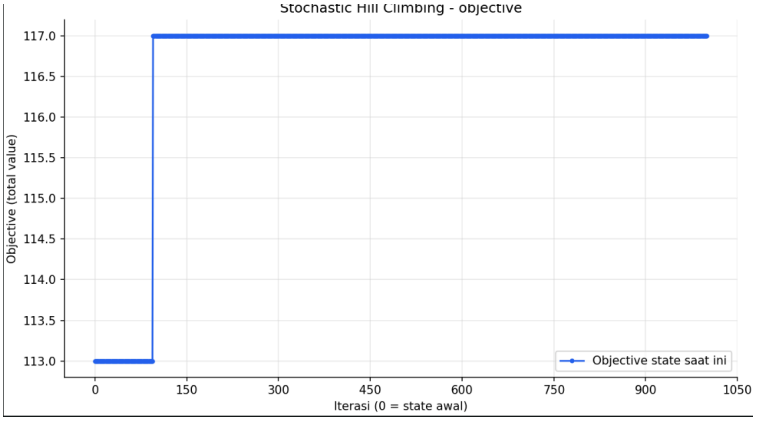


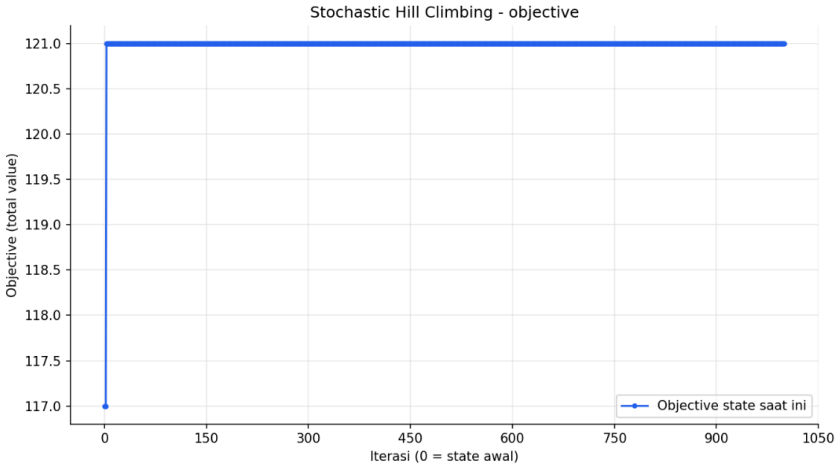


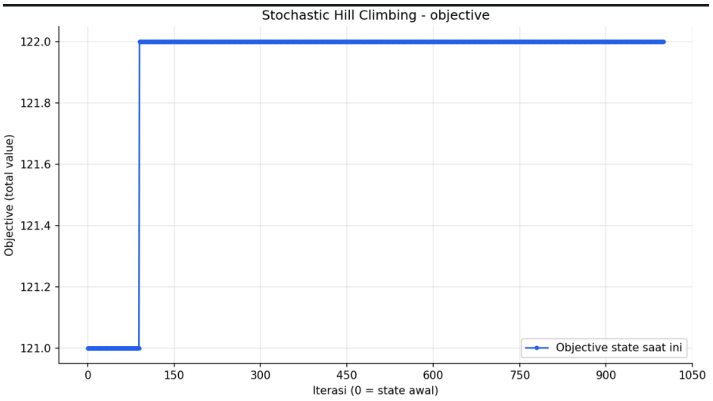


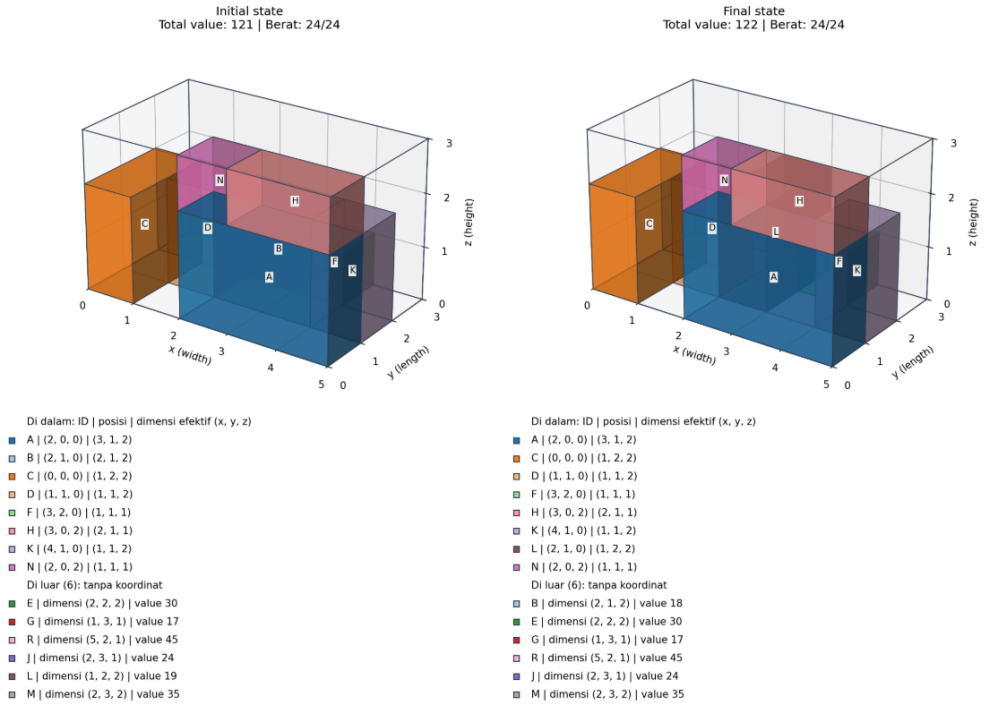
Analisis. Hasil akhirnya identik dengan Steepest Ascent di ketiga run. Setelah mencapai local optimum yang sama, algoritma menghabiskan seluruh jatah 10 sideways move (pindah ke neighbor dengan nilai sama) tanpa pernah menemukan jalan naik, lalu berhenti karena batas sideways tercapai. Artinya local optimum di sini adalah "flat" local maximum (dataran yang luas), bukan shoulder.

F. Analisis Stochastic Hill Climbing



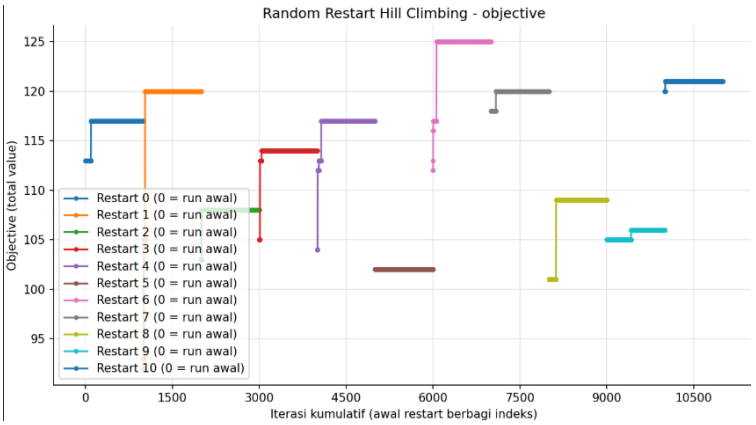


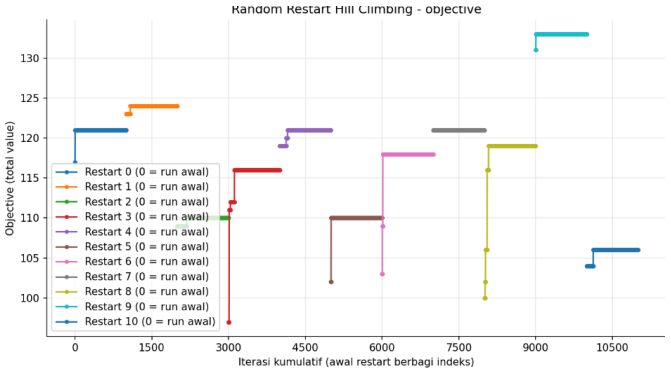


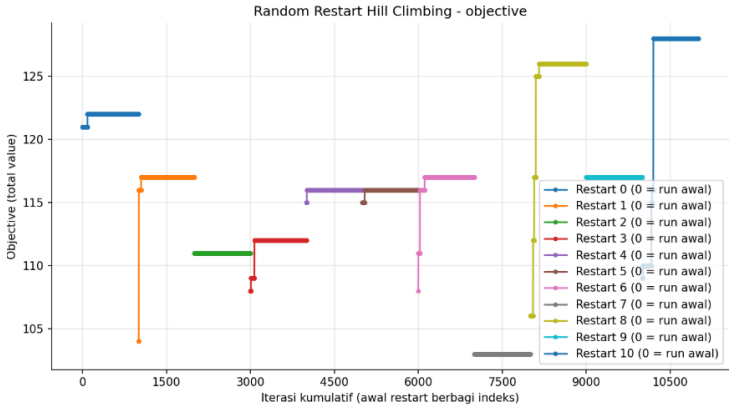


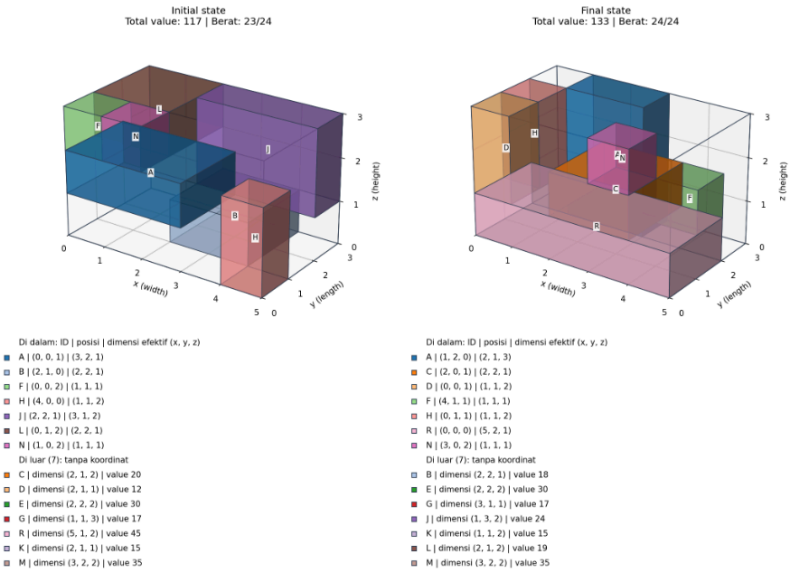
Analisis. Perhatikan bahwa setiap iterasi hanya membangkitkan satu successor acak dan berpindah hanya jika lebih baik, lalu pencarian berhenti setelah *nmax* iterasi. Dari *nmax* iterasi, state hanya berpindah satu kali di setiap run. Neighbor yang dipilih acak didominasi langkah yang tidak memperbaiki nilai, yaitu rotasi dan pergeseran tidak mengubah value, sedangkan memindahkan paket luar ke titik grid acak hampir selalu menghasilkan posisi melayang atau tumpang tindih sehingga ditolak. Neighbor yang benar-benar memperbaiki sangat jarang terambil. Akibatnya hasilnya paling rendah di antara varian HC. Mudah dilihat bahwa pada run 1, Steepest Ascent mencapai 126 karena memeriksa semua neighbor, sedangkan Stochastic HC hanya mencapai 117.

G. Analisis Random Restart Hill Climbing





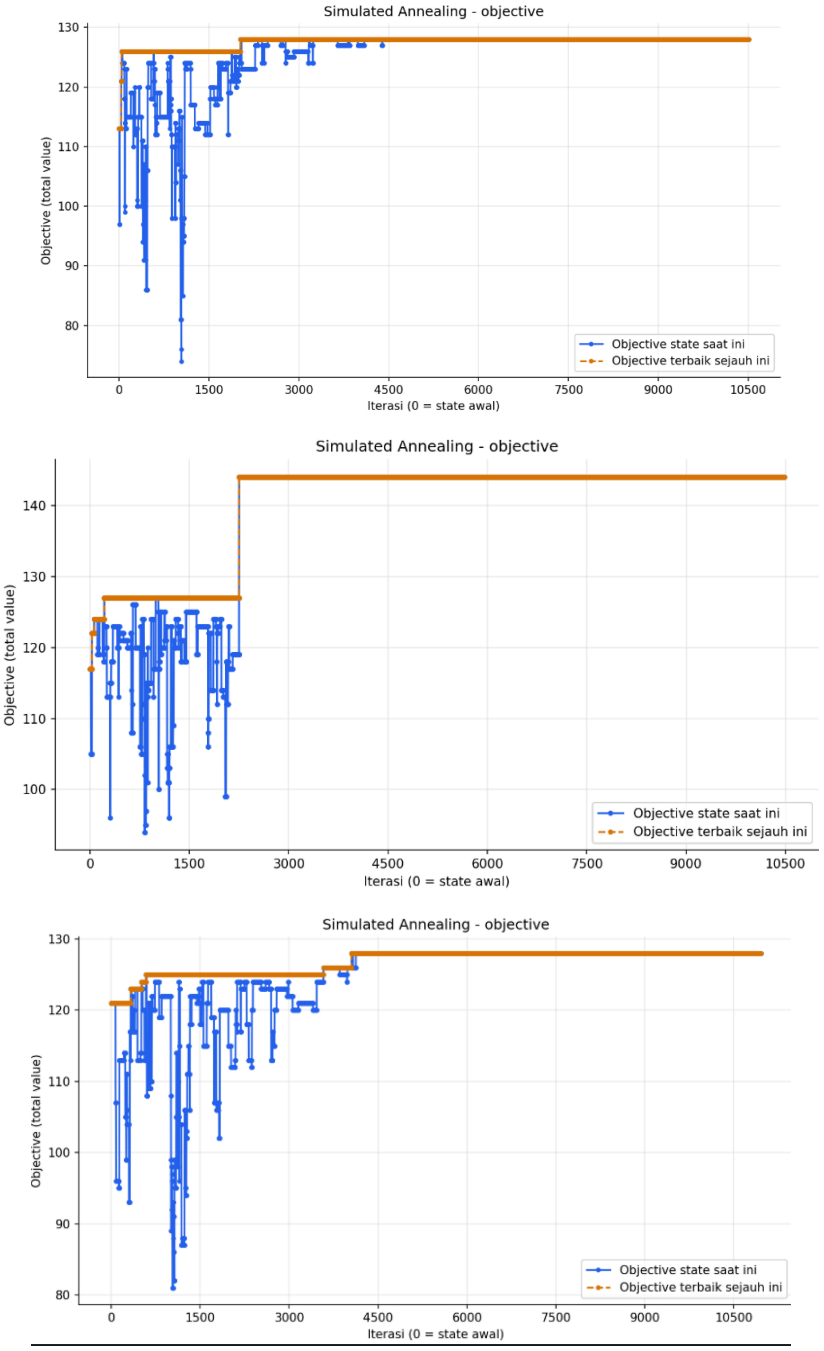




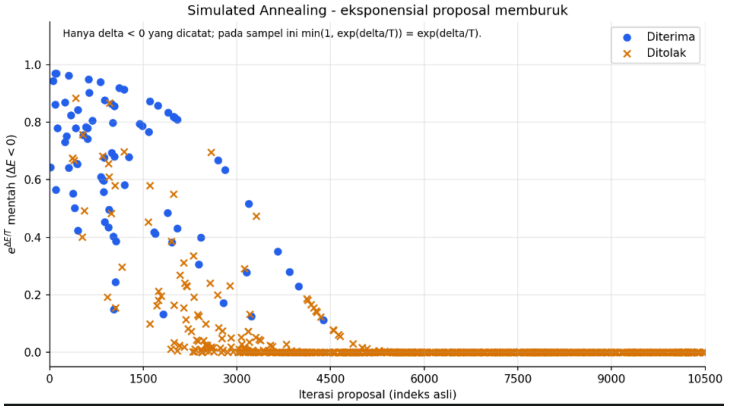
Analisis. Setiap restart dimulai dari initial state acak baru, sehingga local optimum yang dicapai sangat bervariasi. Di ketiga run, hasil terbaik datang dari restart, bukan dari initial state awal. Ini sesuai dengan prinsip dari random restart, yaitu mengulang dari titik acak lain meningkatkan peluang mendarat di bukit yang lebih tinggi. Hasilnya naik dari rata-rata 120 (Stochastic HC tunggal) menjadi 128,67, tetapi dengan biaya sekitar 10 kali lipat durasi. Karena restart terbaik sering muncul di akhir (ke-9 dan ke-10), menambah max_restart kemungkinan masih bisa memperbaiki hasil.

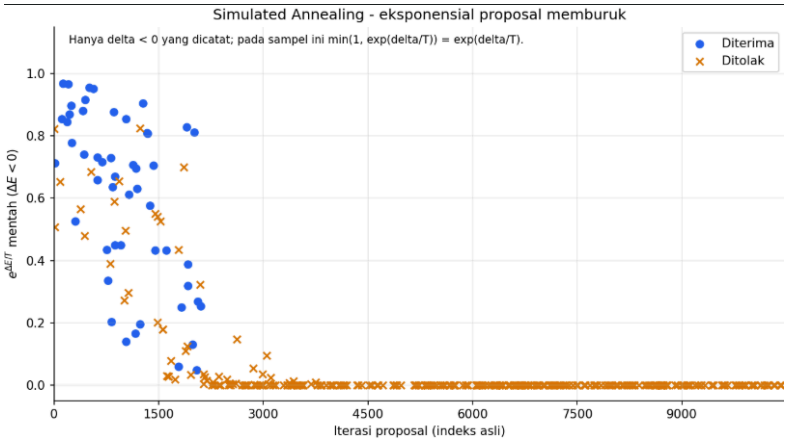
H. Analisis Simulated Annealing

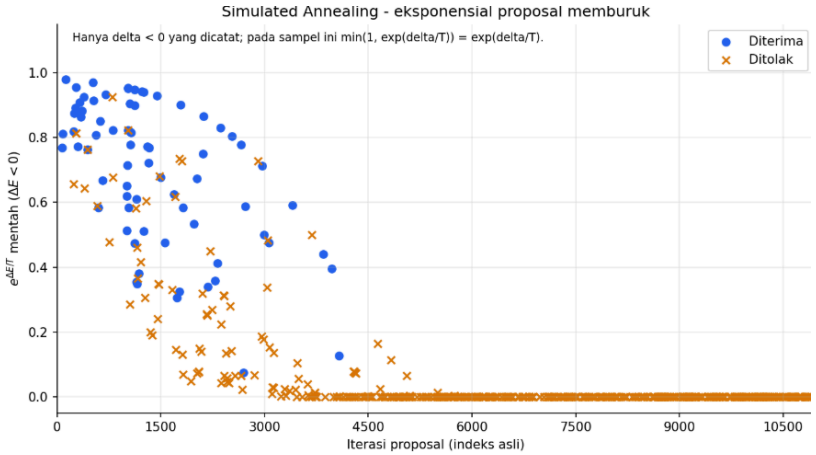
Berikut adalah plot nilai objective terhadap iterasi. Perhatikan bahwa garis biru menyatakan state saat ini, dan garis oranye menyatakan yang terbaik sejauh ini

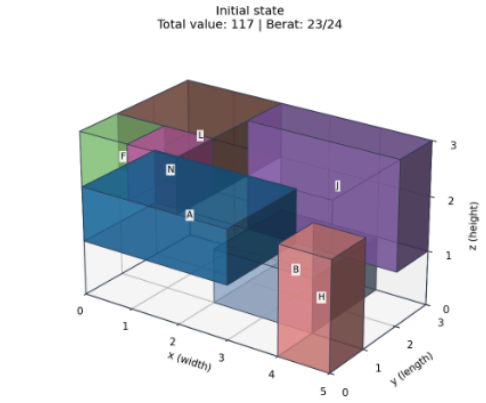


Berikut adalah plot $e^{\frac{\Delta E}{T}}$ terhadap iterasi (hanya langkah dengan deltaE < 0)


Simulated Annealing - eksponensial proposal memburuk
Hanya delta < 0 yang dicatat; pada sampel ini min(1, exp(delta/T)) = exp(delta/T).
Diterima
Ditolak
$e^{\Delta E/T}$ mentah ($\Delta E < 0$)
Iterasi proposal (indeks asli)

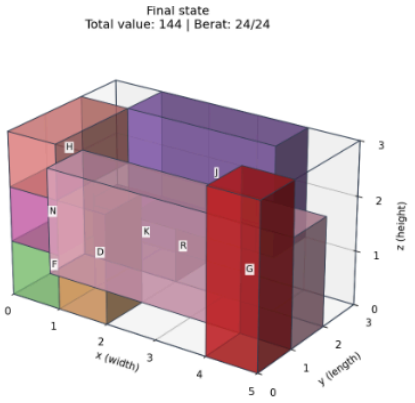

Simulated Annealing - eksponensial proposal memburuk
Hanya delta < 0 yang dicatat; pada sampel ini min(1, exp(delta/T)) = exp(delta/T).
Diterima
Ditolak
$e^{\Delta E/T}$ mentah ($\Delta E < 0$)
Iterasi proposal (indeks asli)


Simulated Annealing - eksponensial proposal memburuk
Hanya delta < 0 yang dicatat; pada sampel ini min(1, exp(delta/T)) = exp(delta/T).
Diterima
Ditolak
$e^{\Delta E/T}$ mentah ($\Delta E < 0$)
Iterasi proposal (indeks asli)



Di dalam: ID | posisi | dimensi efektif (x, y, z)
- A | (0, 0, 1) | (3, 2, 1)
- B | (2, 1, 0) | (2, 2, 1)
- F | (0, 0, 2) | (1, 1, 1)
- H | (4, 0, 0) | (1, 1, 2)
- J | (2, 2, 1) | (3, 1, 2)
- L | (0, 1, 2) | (2, 2, 1)
- N | (1, 0, 2) | (1, 1, 1)

Di luar (7): tanpa koordinat
- C | dimensi (2, 1, 2) | value 20
- D | dimensi (2, 1, 1) | value 12
- E | dimensi (2, 2, 2) | value 30
- G | dimensi (1, 1, 3) | value 17
- R | dimensi (5, 1, 2) | value 45
- K | dimensi (2, 1, 1) | value 15
- M | dimensi (3, 2, 2) | value 35



Di dalam: ID | posisi | dimensi efektif (x, y, z)
- D | (1, 0, 0) | (1, 1, 2)
- F | (0, 0, 0) | (1, 1, 1)
- G | (4, 0, 0) | (1, 1, 3)
- H | (0, 0, 2) | (1, 2, 1)
- R | (0, 1, 0) | (5, 1, 2)
- J | (1, 2, 1) | (3, 1, 2)
- K | (0, 2, 0) | (2, 1, 1)
- N | (0, 0, 1) | (1, 1, 1)

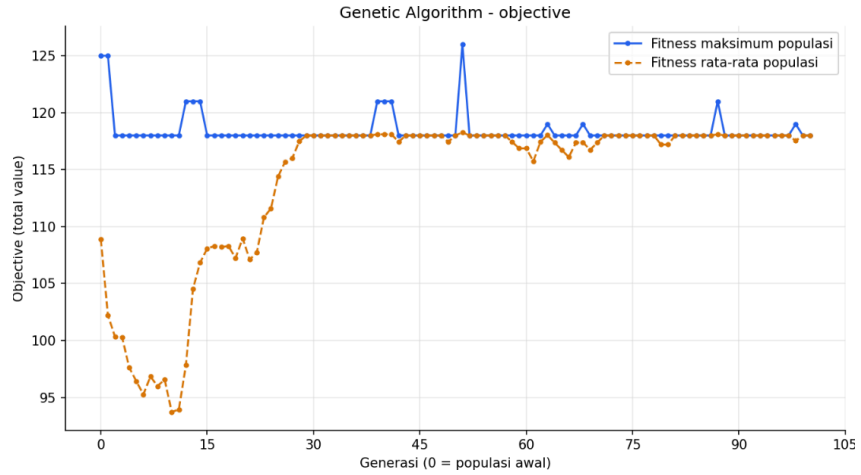
Di luar (6): tanpa koordinat
- A | dimensi (1, 2, 3) | value 25
- B | dimensi (1, 2, 2) | value 18
- C | dimensi (2, 1, 2) | value 20
- E | dimensi (2, 2, 2) | value 30
- L | dimensi (2, 1, 2) | value 19
- M | dimensi (2, 2, 3) | value 35

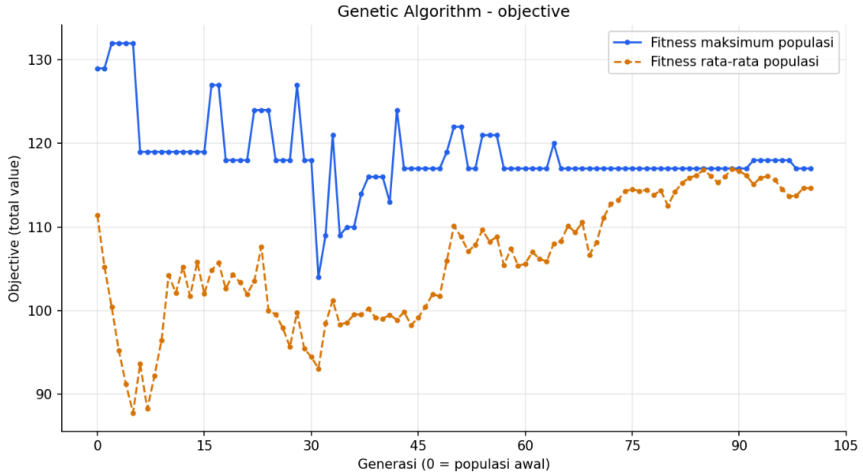
Analisis.

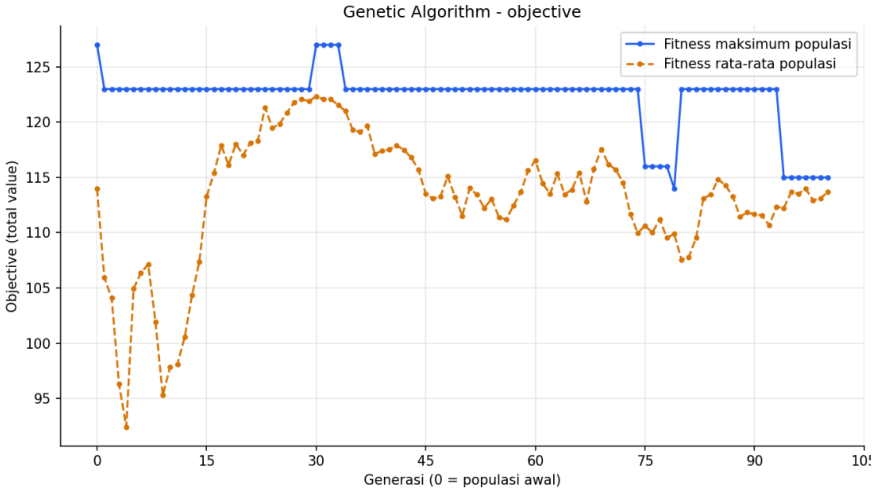
- SA adalah satu-satunya algoritma yang mencapai global optimum.
- Plot e^(deltaE/T) menunjukkan peluang menerima langkah memburuk tinggi di awal, lalu turun mendekati 0. Pada run 2, setelah sekitar iterasi 2.500 hampir semua langkah memburuk ditolak, dan SA berperilaku seperti hill-climbing. Plot objective run 2 memperlihatkan pola yang sama, yaitu nilai state saat ini naik-turun tajam selama ±2.300 iterasi pertama, nilai 144 ditemukan di sekitar iterasi 2.300, lalu tidak berubah lagi.
- Nilai terbaik tercatat tidak membaik selama 200 iterasi berturut-turut sebanyak 2–4 kali per run. Satu periode macet dihitung sekali, termasuk periode panjang di akhir run. Pada run 2, sekitar 8.000 iterasi terakhir berjalan pada suhu yang sudah terlalu rendah untuk menerima langkah turun, sehingga tidak menghasilkan perbaikan.
- Hasil SA bergantung pada urutan acak langkah dan undian penerimaan. Durasi juga paling lama karena iterasinya puluhan kali lebih banyak dari HC.

I. Analisis Genetic Algorithm

a. 3 Run







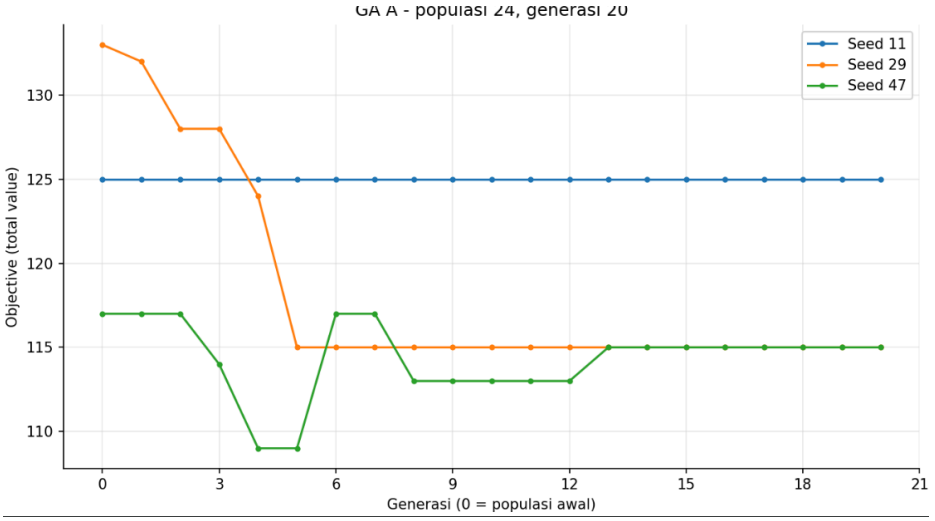
Analisis. Di ketiga run, OF akhir lebih rendah daripada fitness terbaik generasi 0. Beberapa penyebabnya sebagai berikut :

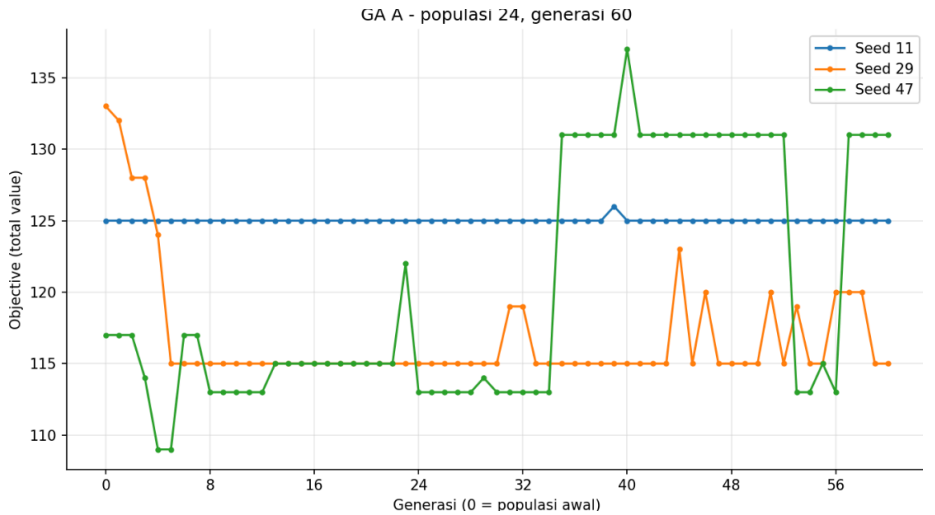
- Seluruh populasi diganti oleh anak setiap generasi, sehingga individu terbaik dapat hilang jika tidak terpilih atau rusak oleh crossover/mutasi.
- Hasil akhir = individu terbaik di populasi terakhir (bukan terbaik sepanjang run), sesuai baris return the best individual in population pada slide.

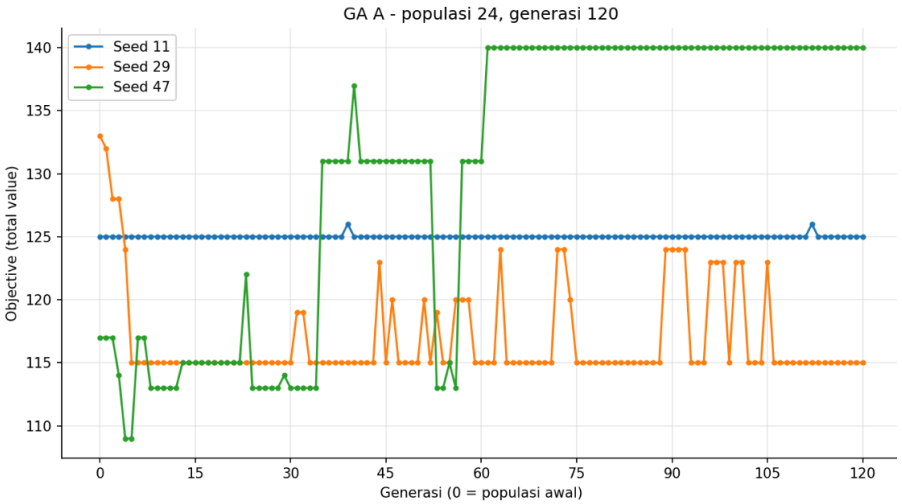
- Seleksi roulette memberi peluang sebanding fitness. Fitness individu berkisar sekitar 100–130, sehingga rasio peluang antar-individu hampir sama dan seleksi mendekati acak.
- Child tidak valid diganti salinan parent. Pada truk sempit, banyak hasil crossover melanggar constraint. Jika gagal setelah beberapa percobaan, anak diganti salinan salah satu orang tua, sehingga keragaman populasi berkurang.

b. 18 Run

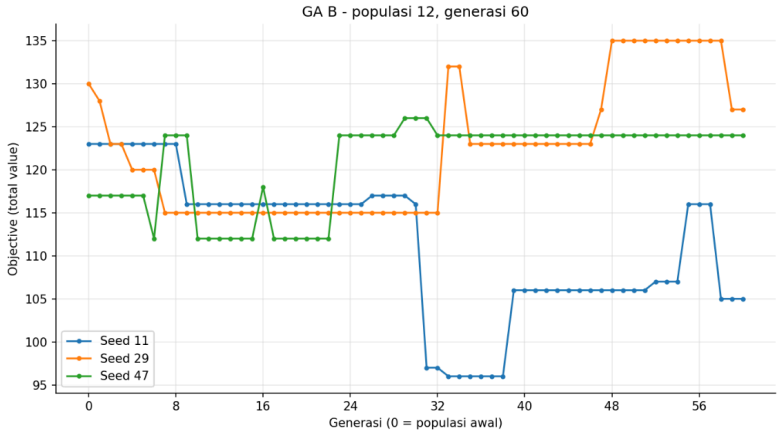
Seri A: populasi tetap 24, generasi divariasikan.

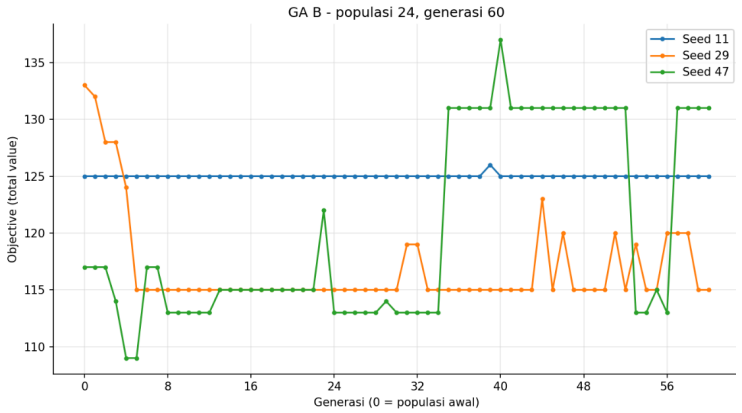


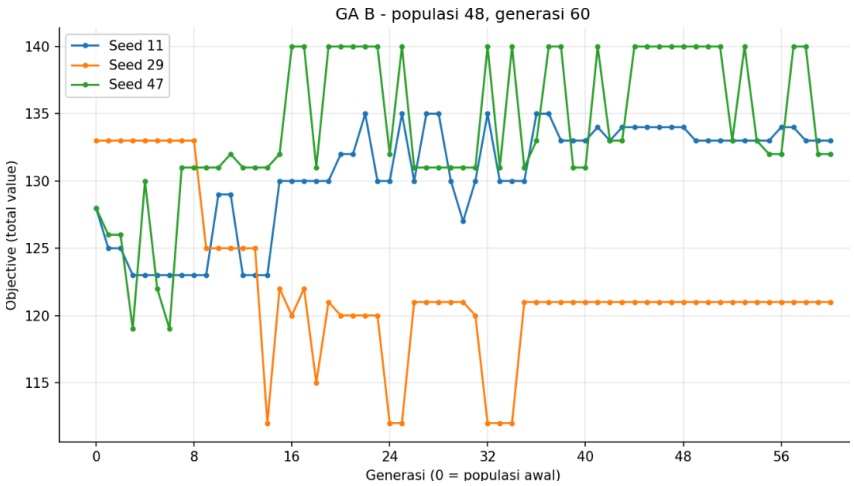




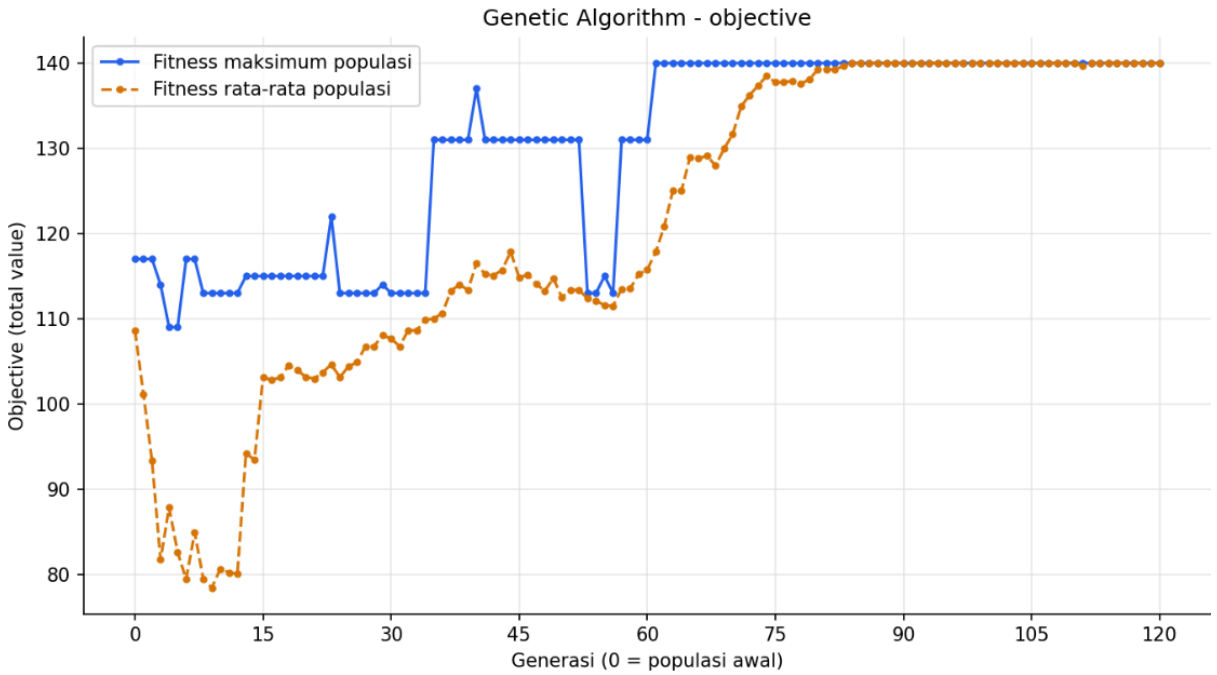
Seri B: generasi tetap 60, populasi divariasikan.

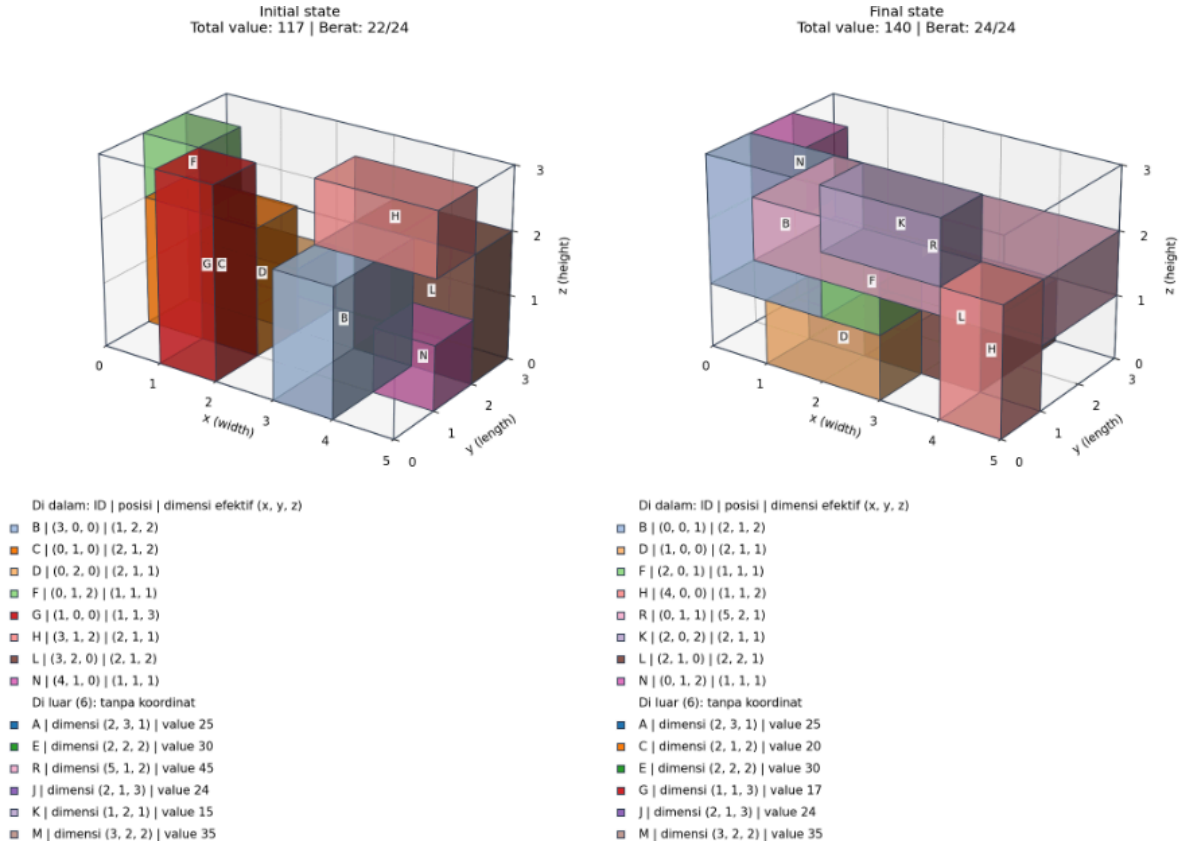






Run terbaik dari 18 run :





Analisis.

- Untuk seri A, rata-rata naik dan durasi naik hampir linear . Namun kenaikan itu seluruhnya berasal dari seed 47; seed 11 tetap 125 dan seed 29 tetap 115. Karena seed sama, ketiga konfigurasi berangkat dari populasi awal yang sama, dan generasi tambahan hanya membantu jika populasi masih punya keragaman untuk dieksplorasi.
- Untuk seri B, rata-rata naik, konfigurasi terbaik di seluruh eksperimen, dengan simpangan baku terkecil di antara variasi populasi. Populasi lebih besar memberi keragaman lebih banyak, sehingga lebih tahan terhadap hilangnya individu baik. Biayanya, durasi naik sekitar 5 kali lipat . Populasi 12 terlalu kecil, bisa dilihat pada seed 11, rata-rata populasi turun dari sekitar 109 ke 86 dan OF akhir hanya 105.
- Konfigurasi kontrol. A_p24_g60 dan B_p24_g60 adalah konfigurasi yang sama (sengaja diulang sebagai kontrol kedua seri) dan memberi hasil identik, menandakan eksperimen dapat direproduksi.
- Kehilangan solusi terbaik. Seed 29 di semua konfigurasi populasi 24 sudah memiliki individu bernilai 133 di generasi 0, tetapi selalu berakhir di 115.
- Hasil terbaik GA adalah 140. Selama run, fitness maksimum sempat menyentuh 140 juga pada B_p48_g60 seed 47 (generasi 16), tetapi tidak dipertahankan sampai akhir.

J. Analisis Hasil

Hanya Simulated Annealing yang mencapai global optimum, dan rata-ratanya juga paling dekat. SA bisa menerima langkah yang memperburuk nilai, sehingga dapat keluar dari local optimum. Ini penting di persoalan ini karena menambah paket sering membutuhkan beberapa langkah persiapan (memindahkan atau mengeluarkan paket lain) yang untuk sementara tidak menaikkan, bahkan menurunkan nilai. Hill-climbing tidak pernah menerima langkah seperti itu, sehingga semua varian HC tunggal terjebak di sekitar 84–88%. Random Restart HC dan GA dengan populasi besar mendekatmelalui keragaman titik awal, bukan melalui langkah turun.

Urutan berdasarkan rata-rata: SA > Random Restart HC ≈ GA populasi 48 > Steepest Ascent = Sideways > Stochastic HC > GA populasi 30.

Sideways move tidak memberi keuntungan apa pun dibanding Steepest Ascent. Local optimum di dataset ini berupa dataran luas yang tidak terlewati dalam 10 langkah menyamping.

Stochastic HC lebih buruk dari Steepest Ascent karena hanya mencoba satu neighbor acak per iterasi, dan neighbor yang memperbaiki sangat jarang terambil.

GA sangat bergantung pada konfigurasi: dengan populasi 48 setara Random Restart HC, tetapi dengan populasi 30 atau 12 justru paling rendah karena kehilangan individu terbaik

Steepest Ascent paling cepat karena berhenti setelah 1–2 iterasi. Stochastic HC dan Sideways HC menyusul. GA berada di tengah . Random Restart HC dan SA paling lama karena melakukan lebih dari 10.000 iterasi. Terlihat trade-off yang jelas: algoritma yang lebih lama berjalan memberikan hasil yang lebih baik, kecuali Sideways HC yang lebih lama tanpa hasil lebih baik.

Varian HC tunggal paling konsisten, tetapi konsisten pada nilai yang rendah. Variasinya hanya mengikuti initial state, karena Steepest Ascent dan Sideways HC bersifat deterministik untuk state awal yang sama. SA paling tidak konsisten. Keacakannya memberi peluang mencapai optimum, tetapi tidak menjaminnya dalam satu run. Random Restart HC berada di tengah. Pada GA, konsistensi bergantung pada konfigurasi.

BAB 3 KESIMPULAN DAN SARAN

Persoalan pengisian peti kemas berhasil dimodelkan sebagai *local search*. State berisi posisi dan orientasi setiap paket, nilai objective adalah total value paket di dalam truk, dan neighbor dibentuk dari tiga langkah sesuai spesifikasi, yaitu menukar dua paket, memindahkan paket, dan merotasi paket 90°. Neighbor yang melanggar constraint selalu ditolak, sehingga setiap state yang dikunjungi selalu valid.

Pada dataset uji (tc.json, 14 paket), global optimum terbukti bernilai 144, dan Simulated Annealing adalah satu-satunya algoritma yang mencapainya, dengan rata-rata tertinggi pula (133,33). Kemampuan SA menerima langkah yang memburuk membuatnya dapat keluar dari local optimum. Sebaliknya, varian Hill-climbing tunggal berjalan cepat (di bawah 2 detik) tetapi mudah terjebak di local optimum dengan rata-rata 120–123. Sideways move tidak memberi perbaikan, sedangkan Random Restart HC mampu memperbaiki hasil hingga rata-rata 128,67 dengan biaya waktu sekitar sepuluh kali lipat.

Genetic Algorithm sangat bergantung pada parameternya. Populasi yang lebih besar memberi hasil yang lebih baik dan lebih konsisten (populasi 48 mencapai rata-rata 128,67), sedangkan menambah jumlah generasi hanya membantu sebagian run. Karena tidak memakai elitisme, individu terbaik dapat hilang di tengah proses, sehingga hasil akhir GA kadang lebih rendah daripada solusi terbaik yang pernah ditemukan. Secara umum, algoritma yang berjalan lebih lama memberikan hasil yang lebih baik, tetapi hasilnya juga lebih bervariasi antar-run.

Untuk pengembangan selanjutnya, GA sebaiknya ditambah elitisme, yaitu mempertahankan individu terbaik di setiap generasi, agar solusi terbaik tidak hilang. Langkah "pindahkan paket" juga dapat dibuat lebih terarah, misalnya hanya memilih posisi yang bertumpu pada lantai atau paket lain, karena memilih titik acak dari seluruh grid membuat sebagian besar langkah ditolak. Parameter SA dapat disesuaikan dengan menaikkan suhu minimum atau mempercepat pendinginan, karena sebagian besar iterasi di akhir tidak menghasilkan perbaikan. Terakhir, eksperimen perlu dijalankan pada dataset yang lebih besar dan dengan lebih dari tiga run per konfigurasi agar perbandingan antar-algoritma lebih dapat dipercaya.

BAB 4 PEMBAGIAN TUGAS

Nama	NIM	Kontribusi
Natanael I. Manurung	13524021	Implementasi & Laporan Hill Climbing, Representasi state dan constraints di laporan.
Manuel Thimoty Silalahi	13524102	Implementasi & Laporan Simulated Annealing. Implementasi Stochastic Hill climbing dan Random Restart Hill Climbing, dan script untuk experiment.
Rava Khoman Tuah Saragih	13524107	Implementasi & Laporan Genetic Algorithm, visualisasi state 3D dan plotting hasil algoritma. Implementasi eksperimen GA dengan variasi jumlah populasi dan generasi, serta objective function alternatif berbasis ETA.

BAB 5 REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video].

Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

BAB 6 LAMPIRAN

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : 1. Natanael I Manurung/13524021
2. Rava Khoman Tuah Saragih/13524102
3. Manuel Thimoty Silalahi/13524107
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas :Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Memeriksa hasil penulisan laporan agar sesuai kaidah penulisan yang baku
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Membantu menyusn kerangka penulisan laporan
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Mempelajari cara kerja algoritma secara lebih mendalam sebelum dikembangkan menjadi kode
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney,	Tidak	Tidak memakai

Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.		
Lainnya, Jelaskan:	Tidak	Tidak memakai

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada
mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Natanael I. Manurung
13524021

Bandung, 20 Oktober 2026



Rava Khoman Tuah Saragih
13524107

Bandung, 20 Oktober 2026



Manuel Thimoty Silalahi
13524102